

Concurrencia e IPC

En un kernel de un SO hay que considerar los problemas generados por la concurrencia.

Dos o más *threads o procesos cooperativos* se comunican mediante el uso de algún recurso en forma compartida. En el caso que dos *threads* compartan datos en su mismo espacio de memoria. Este modelo de comunicación se conoce como *shared memory*. En el caso de procesos en diferentes espacios de memoria (o en diferentes computadoras en una red) será necesario algún mecanismo de *pasaje de mensajes*.

Los mecanismos de sincronización y comunicación ofrecidos por un SO se conoce como *inter-process communication (IPC)*.

En un kernel *reentrante* aún con una única cpu hay que considerar que el código puede tener *interleaving* (diferentes posibles secuencias de ejecución con otro código) producido por el mecanismo de interrupciones.

Lo mismo ocurre en procesos de usuarios en donde el programa puede *capturar* eventos como *señales o excepciones*.

Por ejemplo, sea el siguiente código ejecutándose con las interrupciones habilitadas.

c

```
struct queue q;

void interrupt_handler() {
    ...
    enqueue(q, data);
    ...
}

void enqueue(int value)
{
    struct node * new = new_node(value);
    if (!q.first) {
        q.first = q.last = new;
    }
    ...
}

int dequeue() {
    ...
}
```

```

    if (!q.last)
        // (1)
        error("queue is empty");
    ...
}

```

Si la cpu está ejecutando un thread de un proceso en la línea (1) en `dequeue()` y ocurre una interrupción, el flujo de ejecución se interrumpe y se salta al `interrupt_handler()`. Esta función llama a `enqueue()` y modifica `q.last`. Luego, al retorno de la interrupción, la cpu continuará ejecutando (1), que asume un cierto estado de `q.last` que ha cambiado.

Otro escenario similar ocurre en presencia de *paralelismo* en multiprocesadores. Si dos o más cpus invocan a `enqueue()` cuasi-simultáneamente, podría ocurrir una secuencia de operaciones que rompe el invariante de la estructura de datos.

Un tercer escenario similar ocurre cuando a un thread ejecutando en `enqueue()`, el SO le quita la CPU (en el punto 1, por ejemplo) para darle el control a otro thread que también invoca a `enqueue()` o `dequeue()`.

Concurrencia

Ocurrencia de múltiples *intercalados (interleaving)* de instrucciones debido a paralelismo en multiprocesadores, interrupciones o *thread-switching*.

Cuando existe la posibilidad que dos o más threads intentan modificar un recurso compartido se dice que existe una *condición de carrera* o *race*.

Surge entonces la necesidad de impedir que se ejecuten *trazas* que puedan *romper* invariantes. En el ejemplo anterior, debería asegurarse que las funciones `enqueue()` y `dequeue()` ejecuten al menos parte de su cuerpo sin interrupciones, es decir que no sea posible ejecutar una traza que intercale (*interleaving*) sus operaciones.

Para evitar o tener bajo control estas *paces* se requiere de mecanismos de control de la concurrencia.

Locks

Un *lock* provee *exclusión mutua*, asegurando que cuando un thread adquirió el lock los demás que lo requieran deberán esperar a que el primero lo libere. Esto permite *excluir* trazas no deseadas. Es común el uso del término *mutex* como sinónimo de lock.

Su uso consiste en asociar a cada *recurso compartido* usado por los threads con un *lock* de protección. Cada operación sobre el recurso (ej: en funciones `enqueue()` y `dequeue()`) deberá *adquirir* el lock antes de su acceso y liberarlo luego de su uso.

El uso de locks tiene consecuencias adversas como perder rendimiento porque las operaciones se secuencializan y produce *contención* en los threads que compiten por el recurso.

Retomando el ejemplo anterior, sería posible mantener el invariante de representación de la cola `q` usando locks de la siguiente forma.

c

```
struct queue q;
int lq = 0;

void enqueue(int *value)
{
    struct node * new = new_node(value);
    ...
    acquire(&lq);
    ...
    release(&lq);
}

int dequeue() {
    acquire(&lq);
    ...
    release(&lq);
    return value;
}
```

Listado 1: *Ejemplo de uso de locks.*

Los *locks* producen *contención* (esperas) y que hay que tratar de minimizarlas. En el ejemplo del [listado 1][Listado-1] si el `acquire()` en `enqueue()` estaría por encima de la línea que crea el nuevo nodo, ésta operación secuencializaría la operación de creación, la cual podría ser ejecutada concurrentemente entre varios threads ya que operan sobre valores locales independientes. Lo mismo ocurre si se mueve el `release()` en `dequeue()` a la línea inmediata antes del `return`.

El código protegido por un *lock* se conoce como *región crítica*.

Spinlocks

Un *spinlock* hace que cada thread espere *ocupando* la cpu (en un ciclo). Un primer intento de implementación podría ser el siguiente.

c

```
void acquire(int *lk)
{
    while (*lk)
        ;           // (1)
    *lk = 1;        // (2)
}

void release(int *lk)
{
    *lk = 0;
}
```

Listado 2: Primer intento de implementación de spinlocks.

Sea el siguiente escenario de dos threads usando un lock `l` ;

c

```
int l = 0;

// thread 1                thread 2
...                        acquire(&l);
acquire(&l);                critical region
critical region            release(&l);
release(&l);                ...
...
```

Suponiendo que la cpu está ejecutando el thread 1 dentro de `acquire()` en la línea (1) , sea el valor de `l` cero y antes de la línea (2) ocurre una interrupción de reloj y el kernel decide hacer un *context switch* al thread 2.

Ahora el thread 2 también invoca a `acquire()` y el valor de `l` aún es cero, por lo que ejecutará la línea (2) y continuará su ejecución.

En este punto, el kernel vuelve a hacer un *context switch* al thread 1. El cual continúa en la línea (2) . En este punto, ambos threads están dentro de la región crítica, es decir que no se ha podido garantizar la *exclusión mutua*.

El problema es que el lock en sí mismo es un recurso compartido y las operaciones en `acquire()` y `release()` deberían ejecutarse *atómicamente*. En éste caso se debería impedir que el scheduler puedan realizar un cambio de contexto.

```
void acquire(int *lk)
{
    cli(); // clear (disable or ignore) interrupts
    while (*lk)
        ; // (1)
    *lk = 1; // (2)
}

void release(int *lk)
{
    *lk = 0;
    sti(); // enable interrupts
}
```

Listado 3: *Segundo intento de implementación de spinlocks.*

Solución de Peterson

Peterson propuso en 1981 un algoritmo para resolver el problema de la *región crítica* o *exclusión mutua* basado en un modelo de memoria compartida. La propuesta inicial sólo contempla el problema de la exclusión mutua entre dos threads como se puede ver en el siguiente listado.

```

bool flag[2] = {false, false};
int turn;

lock(i)                                unlock(i)
{                                        {
    flag[i] = true;                      flag[i] = 0;
    other = i==0? 1: 0;                  }
    turn = other;
    while (flag[other] && turn == other) ;
}

// thread 0                            thread 1
lock(0);                                lock(1);
// critical section                    // critical section
// ...                                // ...
// end critical section                // end critical section
unlock(0);                              unlock(0);

```

Listado 4: Solución de Peterson.

Esta solución se basa en usar la variable `flag[i]` para representar la *intención* del thread *i* de ingresar en la región crítica y la variable `turn` para indicar qué thread tendría la prioridad para ingresar.

Si bien es posible extender el algoritmo a N threads, como lo hace el algoritmo *filter*. Este algoritmo funciona sobre N threads identificados por valores $[0..N-1]$ y usa una tabla `levels[N]` donde `levels[i]` representa el *nivel* o *posición* en una cola de espera para ingresar a la región crítica. El nivel cero indica la posición inmediata para entrar en la región crítica.

```

int level[N], last_to_enter[N];
enter(i)
{
    for (int l = 0; l < N; l++) {
        level[i] = l;
        last_to_enter[l] = i;
        while (last_to_enter[l] == i && same_or_higher(i,l))
            ;
    }
}

same_or_higher(i,j)
{

```

```

    for (k=0; k<N; k++) {
        if (k != i && level[k] >= j)
            return true
    }
    return false;
}

leave(i)
{
    level[i] = -1;
}

```

Listado 5: *Algoritmo filter.*

El principal problema es que el algoritmo *filter* no garantiza una espera limitada.

Spinlocks en multiprocesadores

Estas soluciones funcionan en una arquitectura *uniprocador*. En un multiprocesador, las interrupciones se desactivan por CPU, por lo que dos CPUs diferentes podrían ejecutar en paralelo `acquire(&l)` y podrían darse dos trazas paralelas las cuales ambas entrarían en la región crítica.

Si comúnmente implementar un *big lock* por medio de deshabilitar las interrupciones en todo el sistema, esto generalmente es más costoso y puede producir esperas innecesarias.

El principal problema es que en `acquire()` la condición de espera consiste en al menos dos operaciones que pueden ser interrumpidas en el medio.

1. *Test*: Se carga de la memoria y se analiza el valor del lock.
2. *Set*: Se escribe el valor 1 en el lock.

Es necesario lograr estos dos pasos en manera atómica.

Las CPUs modernas proveen instrucciones atómicas para poder implementar locks. Las instrucciones más comunes se conocen como `swap` o `test-and-set`.

El compilador `gcc` provee la función `__sync_lock_test_and_set(&lock, value)` que básicamente es equivalente a la siguiente *operación atómica*:

```

int __sync_lock_test_and_set(int *lock, int value)
{
    int tmp = *lock;

```

```

    *lock = value;
    return tmp;
}

```

En risc-v gcc emite la instrucción `amoswap a5, a5, (s1)` con `a5=1` y `s1=&lock`.

Con esta operación atómica, es posible implementar spinlocks en multiprocesadores.

c

```

void acquire(int *lk)
{
    cli(); // clear (disable) interrupts
    while (__sync_lock_test_and_set(lk, 1) != 0)
        ;
    __sync_synchronize();
    *lk = 1;
}

void release(int *lk)
{
    __sync_lock_release(lk); // *lk = 0;
    __sync_synchronize();
    sti(); // enable interrupts
}

```

Listado 4: Implementación de spinlocks en multiprocesadores.

Además de las dificultades de implementar locks efectivamente hay que tener en cuenta que tanto el compilador como las cpus pueden alterar el orden de las instrucciones generadas o ejecutadas con respecto a cómo que figuran en el código fuente.

Comúnmente esto se hace para mejorar el rendimiento o hacer un mejor aprovechamiento de los registros.

Esto se hace en instrucciones que se consideran *independientes*, es decir, que no tienen dependencias entre sus operandos.

En estos casos es necesario instruir al compilador y al hardware que no reordene.

Además en los multiprocesadores modernos comúnmente se usan *modelos de memoria relajados*, en el cual una escritura en memoria por una cpu puede no ser visible en otra. El objetivo es reducir el número de acceso a RAM y aprovechar los datos en las cachés locales a cada cpu (*caché L1*).

Esto puede ser un problema para implementar estas primitivas ya que si un cambio en el valor del lock no es visible por otra cpu, no funcionaría.

Estas arquitecturas ofrecen comúnmente una instrucción que actúa como una *barrera de memoria* o *fence*, que al ejecutarse, las escrituras se deben reflejar en las otras cpus (generalmente invalidando las líneas de caché locales a cada cpu). Generalmente estas instrucciones también indican que cualquier reordenamiento no debe cruzar un *fence*.

En gcc la función `__sync_synchronize()` actúa como una *barrera de memoria*. En la plataforma risc-v gcc genera la instrucción *fence*.

Ver la implementación de *spinlocks* dados en el paso *02-hello-smp* de *eos-steps*.

Sleep locks

Un spinlock sólo es útil para realizar esperas cortas, es decir cuando la región crítica toma muy poco tiempo ya que se está usando la CPU en la espera.

En el caso que la espera dependa de una condición que puede cambiar dentro de demasiado tiempo es preferible *suspender* la ejecución del thread o proceso y reiniciarlo cuando la condición haya ccambiado. Esto es típico cuando en una región crítica se deben hacer operaciones de entrada-salida o computaciones largas.

Un *sleep lock* realiza esta tarea. Son similares a las *variables de condición*, las cuales asocian un conjunto de procesos a una condición de espera (el lock). Una API de sleep locks podría ser de la siguiente forma.

- `void sleep(task* t, wait_queue* q)` : Pasa `t` a estado `SLEEPING` dejándolo encolado en la cola `q`.
- `task* wakeup(wait_queue* q)` : Desencola una tarea de `q` pasándola a estado `RUNNABLE`.

Obviamente, la implementación de estas primitivas requiere el uso de primitivas de locks como los *spinlocks* ya que la cola es un recurso compartido y `sleep` y `wakeup` pueden ejecutarse concurrentemente.

Uso de locks

El uso de locks debe ser cuidadoso porque pueden generar problemas como *deadlock*.

Uno de los principales problemas es cuando el uso de locks no está encapsulado en una única función, sino que una lo adquiere y se debe liberar en otra. Esto muestra que *los locks no son modulares*. El uso de locks crea efectos colaterales que *deben ser documentados*.

Si un thread intenta adquirir un recurso previamente *adquirido* por él mismo, se bloqueará por siempre (*self lock*).

Este problema puede resolverse implementando *locks recursivos*.

Otro problema es el uso de múltiples locks. Suponiendo que se usan locks como se muestra a continuación.

c

```
f() {                                g() {
    ...                               ...
    acquire(&l1);                     acquire(&l2);
    ...                               ...      (1)
    acquire(&l2);                     acquire(&l1);
    ...                               ...
    release(&l2);                     release(&l1);
    ...                               ...
    release(&l1);                     release(&l2);
}
```

Listado 2: *Uso de locks.*

En este caso hay un potencial *deadlock*. Ya que un thread ejecutando `f()` en el punto (1) y otro thread ejecutando `g()` en el mismo punto quedarán esperándose mutuamente (*deadlock*).

Este escenario produce una *espera circular*. La forma de evitarlo es siempre manteniendo el mismo orden en el uso de los locks. En un sistema complejo esto puede ser muy difícil de lograr.

Deadlock

Un **deadlock** entre un grupo de procesos ocurre cuando se dan las siguientes condiciones simultáneamente:

1. **Exclusión mutua:** Los procesos han adquirido un recurso en forma exclusiva.
2. **Espera y retención:** Un proceso retiene un recurso mientras espera por otro.
3. **No quita (preemption):** Los procesos sólo liberan sus recursos voluntariamente.
4. **Espera circular:** Existe una dependencia circular de recursos. Sean los procesos $P_0, P_1, \dots, P_{n-1}, P_i$, para todo i tal que $0 \leq i < n$, espera por un recurso adquirido por $P_{(i+1) \bmod n}$.

Generalmente en un SO se debe aplicar una disciplina particular en el uso de locks. Por ejemplo, se debe definir el *orden de uso* de locks en todos los módulos del sistema.

Semáforos

Un semáforo comúnmente representa una cantidad de recursos disponibles. Es una primitiva de sincronización más general que los locks.

Un semáforo *n*-ario (o *counting semaphore) permite que hasta *n* threads accedan a recursos compartidos. Cualquier thread adicional que intente acceder al recurso deberá esperar hasta que algún otro lo libere.

Comúnmente un semáforo se implementa con un contador más un sleeplock. Una API de semáforos generalmente define al menos las siguientes operaciones:

1. `sem_init(s, value)` : Inicializa el valor del semáforo `s` .
2. `sem_wait(s)` : Si el valor de `s` está en cero, suspende al invocante. Sino, decrementa el valor del semáforo `s` .
3. `sem_signal(s)` : Incrementa el valor de `s` y *despierta* a los threads o procesos bloqueados en `s` .

Es fácil de ver que si $n = 1$ sólo 1 thread podrá acceder al recurso garantizando exclusión mutua, lo que lo hace equivalente a un *mutex*.

El siguiente programa muestra el ejemplo clásico del problema del *productor-consumidor* con semáforos. El problema consiste en que hay un (o más) productor que produce valores en un buffer acotado de capacidad *N* y uno o más consumidores que extraen valores del buffer y los procesan.

Los productores y consumidores se ejecutan concurrentemente. Comúnmente el buffer se maneja como una cola (disciplina FIFO).

c

```
buffer buf[N];
semaphore mutex = 1,
           full = N,
           empty = 0;

void producer()
{
    while (true) {
        value = gen_value();
        sem_wait(&full);
        add_to_buffer(value);
        sem_notify(&empty);
    }
}

void consumer()
{
    while (true) {
        sem_wait(&empty);
        value = get_from_buffer();
        sem_signal(&full);
        process(value);
    }
}
```

```

void add_to_buffer(value)                item_type get_from_buffer()
{
    sem_wait(&mutex);                    {
    // add value to buffer                sem_wait(&mutex);
    sem_signal(&mutex);                  // extract item from buffer
}                                        sem_signal(&mutex);
}
```

Listado 3: *Productor-consumidor con semáforos.*

El semáforo `full` representa el número de valores en el buffer y el semáforo `empty` representa el número de slots libres en el buffer.

Las operaciones sobre el buffer usan un semáforo binario o `mutex` para asegurar sus invariantes de representación ante la concurrencia.

Monitores

Como ya se analizó, el uso de locks y semáforos no es simple. El principal problema es que no son modulares. En 1973 Hansen y Hoare en 1974 propusieron un mecanismo de alto nivel llamado *monitor*. Un *monitor* es una construcción sintáctica de un lenguaje de programación que *encapsula* datos y las operaciones sobre ellos. Los procesos o threads sólo pueden manipular y acceder a los datos sólo por medio de las operaciones del monitor. Es posible ver un monitor como un *objeto*, *clase* o *módulo* que es *thread safe*, es decir que los invariantes de sus operaciones están protegidos en presencia de concurrencia.

Su implementación común es usar *variables de condición* y al menos un *mutex* para asegurar la *exclusión mutua* entre las operaciones concurrentes sobre los mismos datos. Una *variable de condición* representa una *cola de threads* asociada a un *mutex* interno esperando a que una condición *c* se torne verdadera. Comúnmente se implementa con dos operaciones `wait(c)` y `signal(c)`.

El siguiente listado muestra el ejemplo del productor-consumidor usando un monitor en un lenguaje hipotético:

```

monitor PC {
    condition full, empty;
    data buffer[N];
    int count = 0;

    void put(data item) {
        if (count == N) wait(full);
```

```

        insert(buffer, item);
        if (++count == 1) signal(empty);
    }

    data get() {
        if (count == 0) wait(empty);
        data item = remove(buffer);
        if (--count == N-1) signal(full);
    }
};

void producer() {
    while (true) {
        data item = gen_item();
        PC.put(item);
    }
}

void consumer() {
    while (true) {
        data item = PC.get();
        process(item);
    }
}

```

Listado 4: *Productor-consumidor con un monitor.*

Algunos lenguajes con soporte de concurrencia soportan monitores. El lenguaje Java permite definir un monitor mediante el uso de *métodos sincronizados*. Una clase que declara métodos con el modificador `synchronized` asegura que su ejecución no tendrá *interleavings* con otros métodos sincronizados sobre el mismo objeto.

Java tiene las operaciones `wait()` y `notify()` que operan sobre el monitor del objeto. Son similares a `wait(channel)` y `wakeup(channel)` ya vistas en donde `channel` es implícito (o equivalente a `this`).

También Java provee `notifyAll()` que *despierta* a todos los threads esperando en el monitor (`notify()` despierta sólo uno seleccionado aleatoriamente).

El uso de monitores permiten escribir programas concurrentes de una forma mas modular. Esta idea se puede aplicar en lenguajes sin monitores, usando una disciplina de modularización equivalente, *encapsulando* y *ocultando* el uso de locks.

Mensajes

Los mecanismos analizados anteriormente se basan en el modelo de concurrencia basado en *shared memory*. En el caso que dos o más procesos (cada uno ejecutando en su propio espacio de memoria) quieran comunicarse deberán usar algún mecanismo de *pasaje de mensajes*.

El modelo *message-passing* se basa en al menos dos primitivas del tipo

- `send(destination, message)`
- `receive(source, &message)`

La operación `send` envía datos a un proceso identificado como `destination` mientras que `receive` recibe un mensaje del emisor `source` (que puede ser `any`).

Estos mecanismos permitirían comunicar procesos locales o remotos (entre procesos ejecutándose en diferentes computadoras conectadas en red).

La comunicación puede implementarse usando *buffers de mensajes* o *mailboxes* o sin ellos.

En el primer caso la comunicación se dice *asincrónica* ya que el emisor no tiene que bloquearse ya que el mensaje quedará en el *mailbox*. Un receptor sólo se bloqueará si el *mailbox* está vacío.

Si no se usan buffers, la comunicación será *sincrónica* ya que los procesos deberán *coincidir* (esperarse) en las operaciones `send/receive` . Esto se conoce como un *rendezvous*.

En un SO con diseño *micro-kernels* como [MINIX](#), los subsistemas (gestor de procesos, memoria, sistemas de archivos, etc) se implementan como *servers* que reciben requerimientos de los demás subsistemas por medio de un mecanismo de mensajes y tienen la siguiente forma:

```
int main(void)
{
    message m;
    init_subsystem();
    while (TRUE) {
        receive(ANY, &m);
        switch (m.type) {
            case REQUEST_1:
                ...
                send(other_subsystem, m2);
                ...
                break;
            ...
        }
    }
}
```

```
}  
}  
}
```

Es común que un sistema basado en pasaje de mensajes ofrezca operaciones de alto nivel basados en diferentes *patrones de comunicación* para *grupos de procesos* como los siguientes:

- `broadcast(sender, receivers, data)` : Envía un dato a un conjunto de procesos.
- `scatter(sender, receivers, array)` : Distribuye partes del `array` de datos a los receptores.
- `gather(receiver, senders, &array)` : El receptor reúne en `array` las partes de datos enviados por los emisores.
- `value = mapreduce(sender, receivers, op, array)` : El emisor distribuye (*map*) los datos a los receptores. Cada uno aplica la operación `op` sobre los datos recibidos (en paralelo). El emisor recibe los valores computados y también aplica la operación `op` para producir el valor final.

La operación `op` debe ser asociativa y conmutativa para poder explotar el máximo paralelismo.

- `barrier(group)` : Actúa como una *barrera de sincronización*. Los procesos pueden continuar luego que todos ejecuten (alcancen) la barrera.

El estándar [Message-Passing Interface \(MPI\)](#) define una API con operaciones de este tipo. Es ampliamente usado en redes de computadoras o *clusters* y permite *abstraer* la red como un sistema multiprocesador para realizar *computación de alto desempeño* (*high performance computing* (HPC)).

Remote procedure call

El mecanismo de *invocación a procedimientos o funciones remotas* o *RPC* abstrae el mecanismo de pasaje de mensajes subyacente para que un programador en lugar de usar primitivas del tipo `send/recv` realice un pedido de un servicio a un proceso remoto usando el concepto familiar de *invocaciones a funciones*.

Un *servidor* ofrece una API en forma de un conjunto de tipos de datos y funciones. Esto comúnmente se define en un *interface definition language* (IDL) para independizarse del lenguaje de programación de implementación.

```
interface memory {
    int malloc(int size);
    int free(void *address);
}
```

Listado 5: *Interface de ejemplo de un memory allocator.*

Un *IDL compiler* genera desde la interface los *stubs* para el cliente y el servidor en un lenguaje de programación específico.

```
rpc_server s = rpc_get_server();
void* malloc(int size)
{
    rpc_call m = {.f = MALLOC_MSG, .args = [rpc_marshall(size), 0]};
    send(rpc_server, m);
    rpc_response r;
    receive(rpc_server, &r);
    return rpc_unmarshall(r.result);
}

void free(void *address)
{
    ...
}
```

Listado 6: *Ejemplo de un stub del cliente.*

El programa cliente se enlaza con el *client stub* y podrá invocar al servicio remoto simplemente invocando a `data_ptr = malloc(sizeof data);` .

```
extern void * malloc(int);
extern void free(void *);

void dispatcher(void) {
    while (true) {
        rpc_call m;
        receive(ANY, &m);
        switch (m.f) {
            case MALLOC_MSG:
                void * result = malloc(rpc_unmarshall(m.args[0]));
                rpc_response r;
```



```

        r.result = rpc_marshall(result);
        send(m.sender, r);
        break;
    case FREE_MSG:
        ...
        break;
    }
}
}

```

Listado 7: Ejemplo de un stub del servidor.

El *server stub* se enlaza con el programa servidor (el cual debe implementar las funciones de servicio de la API) y consiste básicamente en un dispatcher que recibe los mensajes que representan invocaciones remotas de los clientes e invoca a las funciones que implementan cada servicio.

Las operaciones de *marshalling* y *unmarshalling* serializan y deserializan datos. La serialización codifica un dato en alguna forma (binaria o textual) para ser transmitido. La deserialización es el proceso inverso: Obtiene el valor recibido en un mensaje y se representa como un dato de un determinado tipo en el lenguaje de programación usado.

Por ejemplo, un valor de tipo entero puede representarse en una estructura de datos de la forma `{type: INT, value: 5}` para transmitirse como un parámetro o un resultado en una invocación remota.

Futuros y promesas

Un *futuro* o *promesa* refiere a una abstracción que representa un valor que será computado por algún otro componente (thread) de ejecución concurrente. Se usan comúnmente para implementar *data-flow variables* en entornos concurrentes: Variables que ante una lectura y ésta no tiene asignado un valor, el thread debe "esperar" por que otro le asigne algún valor. Las operaciones de lectura actúan como un mecanismo de sincronización.

En este modelo es común que se dispongan de las siguientes operaciones:

- `future f = async g(args)` : Invocación asincrónica de `g()` . El invocante continúa con la ejecución. Su implementación comúnmente ejecuta `g` en un nuevo thread. Retorna un *future*: una variable *data-flow*.
-

`value = f.get()` : Esta operación, comúnmente ejecutada por el invocante de un `future f = async g(...)` , obtiene el valor (encapsulado en `f`) posiblemente bloqueando al invocante hasta que el valor sea computado por `g()` .

Este mecanismo es *built-in* en algunos lenguajes de programación como en [Oz](#) en los cuales la operación `get()` es implícita. En otros lenguajes como en C++, se implementan en bibliotecas y el acceso a un *future* es explícito (como `f.get()`).

Concurrencia sin locks (lock free)

Una estructura de datos que puede usarse concurrentemente y que no usa operaciones de bloqueo se denominan *lock-free*. Obviamente su ventaja es que es posible mejorar el rendimiento de un sistema y seguridad: Si un thread *finaliza* con un lock adquirido, los demás threads intentando acceder no podrán progresar.

Una estructura de datos *wait-free* si cada thread que la usa puede completar sus operaciones en un número acotado de pasos.

Así, un *spinlock* califica como *lock-free* pero no como *wait-free*. El diseño de estas estructuras de datos no es simple. Su implementación generalmente requiere de instrucciones atómicas del tipo `compare_and_swap(&target, old, new)` la cual realiza los siguientes pasos atómicamente.

1. Compara los valores de `target` con el valor `old` .
2. Si son iguales, a `target` le asigna `new` y retorna `true` .
3. Sino, retorna `false` .

El [listado 8](#) muestra un fragmento de una lista enlazada *lock-free*.

```
/* insert at front */
push(list* head, T value)
{
    list_node * n = create_node(value);
    n->next = head->first; // (1)
    while (!compare_and_swap(&(head->first), n->next, n)) ; // (2)
}
```

Listado 8: Ejemplo de `push()` *lock-free*.

La operación atómica `compare_and_swap()` permite verificar si otro thread modificó `head->first` entre las líneas (1) y (2), en cuyo caso la operación `head->first = n` debería *reintentarse*.

La estrategia consiste en identificar los puntos de programa en los que puede haber una *race condition* y asegurarse que se logró la actualización preservando el invariante. Se debe notar que es muy difícil de lograr cuando están involucrados varios pasos.

IPC en UNIX

Los SO tipo UNIX se caracterizan por ofrecer una buena variedad de mecanismos de sincronización y comunicación entre procesos como los que se listan en la siguiente tabla.

Mecanismo	Llamadas al sistema
File	<i>open, read, write, close, ...</i>
Pipes	<i>pipe, read, write, close, ...</i>
Named pipes	<i>mkfifo, read, write, close, ...</i>
Signals	<i>kill, signal, alarm, ...</i>
Sockets	<i>listen, connect, accept, send, recv, ...</i>
Message queues	<i>msgget, msgrcv, msgctl, ...</i>
Semaphores	<i>semget, semctl, semop, ...</i>
Shared memory	<i>shmget, shmat, shmctl, ...</i>
Memory mapped files	<i>mmap, munmap, ...</i>

Tabla 1: *Mecanismos de IPC en sistemas UNIX.*

Todos estos mecanismos permiten la comunicación y sincronización entre procesos locales (en una misma computadora) a excepción de los *sockets* que permiten comunicar procesos en diferentes computadoras (nodos) de una red.